

SOFTWARE REQUIREMENTS SPECIFICATION

NomachiBot — Debt Agreement Telegram Mini App

MVP Build — Technical Review · SRS · Step-by-Step Roadmap

Version	1.0 — Initial Specification (built from zero)
Date	May 2026
Target Platform	Telegram Mini App + standalone web
Primary Market	Uzbekistan (scalable globally)
Stack	React · Node.js · PostgreSQL · Puppeteer
Key Decisions	Borrower initiates · lender logs repayments · no interest · witnesses · open public ledger · optional selfies · 3 languages (UZ / RU / EN)

PART 1 — Software Requirements Specification (SRS) — NomachiBot v1.0

1.1 Project Overview

Product Name	NomachiBot — Debt Agreement Telegram Mini App
Product Type	Telegram Mini App + publicly accessible web
Core Purpose	Enable individuals to create, countersign, and publicly track a zero-interest debt agreement, entirely inside Telegram.
Participants	Borrower (initiates) · Lender (confirms) · up to 3 optional Witnesses
Primary Market	Uzbekistan; architecture ready to scale to CIS and beyond
MVP Scope	Contract creation, optional selfies per participant, PDF generation, QR code, public live ledger, downloadable updated PDF.

1.2 Roles and Participants

Role	Who	Key Actions
Borrower	Person receiving money	Initiates contract. Enters all terms. Optionally takes selfie. Generates invite links for lender and witnesses.
Lender	Person giving money	Opens invite link. Reviews terms. Enters own name / optional details. Optionally takes selfie. Confirms contract. Logs all repayments received from the borrower.
Witness 1–3	Optional observers	Open witness invite link. Enter own name / optional details. Optionally take selfie. Confirm their participation. No financial role.
Guest	Anyone with the QR	Scans QR code. Views full contract info and repayment history. No login required. Can download the current PDF.
Admin	System operator	Internal panel only. View contracts, delete flagged content, user management. Not a product feature.

1.3 Goals and Non-Goals for MVP

Goals — MVP must have

- Borrower creates a contract by entering: their own mandatory identity fields, lender's mandatory identity fields, total debt amount, monthly repayment amount.
- Real-time calculator: as borrower types the monthly amount, the number of months updates instantly. No interest, no fees, no financial calculation beyond $\text{months} = \lceil \text{total} \div \text{monthly} \rceil$.
- Borrower optionally adds up to 3 witnesses by name (and their Telegram username if known).
- System generates unique shareable invite links: one for the lender, one for each witness.
- Lender opens their link, reviews terms, enters/confirms their own details, optionally takes a selfie, and confirms the contract.
- Each witness independently opens their link, enters/confirms their own details, optionally takes a selfie, and confirms.

- Once the lender confirms, the contract status changes to 'active' and the PDF is generated.
- PDF layout: each participant has their own info block (name, patronymic, Telegram username, phone, address) with their selfie photo displayed directly beside it — or a placeholder if no selfie was taken.
- A QR code is printed at the BOTTOM of the PDF, linking to the public status page.
- Public status page: accessible by anyone without login. Shows all contract details, all participant info, full repayment history as logged by the borrower.
- Download button on the public page regenerates and downloads a fresh PDF with all current payment updates and the QR code at the bottom.
- Lender logs repayments as they are received from the borrower. Each repayment entry is shown on the public page and included in the downloadable PDF.

Non-Goals — out of MVP scope

- No interest, compound interest, late fees, or any financial calculation other than $\text{months} = \text{ceil}(\text{total} / \text{monthly})$.
- No server-side face recognition or biometric matching — Phase 2.
- No liveness detection — Phase 2.
- No payment processing or bank integrations.
- No blockchain or smart contracts.
- No native iOS/Android apps.
- No automated payment reminders — Phase 2.

1.4 Identity Fields per Participant

i These rules apply equally to the Borrower, the Lender, and each Witness.

Field	Required?	Notes
First name	MANDATORY	Text input. Min 2 characters.
Last name	MANDATORY	Text input.
Telegram ID	MANDATORY	Auto-populated from Telegram account for the authenticated user. For lender / witnesses entered by borrower: @username text field (Telegram will validate at confirmation step).
Patronymic name	Optional	Text input. Common in Uzbek/CIS naming conventions.
Phone number	Optional	Formatted input (+998 prefix suggested for Uzbekistan). Validates E.164 format.
Address	Optional	Free-text textarea. City, street, apartment.
Selfie photo	Optional	Native camera input (<input capture='user'>). If not taken, PDF shows a neutral avatar placeholder in the photo position.

1.5 The Debt Calculator — Functional Specification

The debt calculator is the core UI element on the contract creation screen. It has no interest rate field. The formula is:

$$\text{Number of Months} = \text{CEILING}(\text{Total Debt Amount} \div \text{Monthly Repayment Amount})$$

Example: \$1 000 total ÷ \$100/month → 10 months. \$1 000 ÷ \$300/month → 4 months (3.33 rounds up).

Real-Time Behaviour

- Both fields are number inputs: Total Debt Amount and Monthly Repayment Amount.
- Number of Months is a read-only display — it is never entered by the user.
- Every time either input changes (on every keystroke), the months value recalculates instantly using JavaScript `Math.ceil(total / monthly)`. No API call is made.
- If monthly > total: display '1 month' (one full payment covers the debt).
- If monthly is 0 or empty: display '—' (not calculated yet).
- The result is shown prominently: large font, highlighted box, next to the two input fields.
- A summary line below shows: 'Last payment: \$X' where X = total mod monthly (or full monthly if evenly divisible).

Calculator UI Wire

Total Debt Amount: [\$ 1,000.00]		
Monthly Repayment: [\$ 100.00]		
10 months to repay ← live update		
Last payment: \$100.00		

1.6 Contract Creation Flow (Borrower Initiates)

Step 1 — Borrower fills the contract form

- Section A — My details (Borrower): mandatory fields pre-populated from Telegram. Borrower can edit first/last name, add patronymic, phone, address, take selfie.
- Section B — Lender details: borrower enters lender's Telegram @username (mandatory). First/last name mandatory. Patronymic, phone, address optional.
- Section C — Witnesses (optional): borrower can add 1, 2, or 3 witnesses. Each witness needs at minimum a first name, last name, and Telegram @username.
- Section D — Debt Terms: Total Amount, Monthly Repayment Amount, Currency (UZS / USD / EUR), Start Date, optional Description/Purpose of loan.
- Section E — Repayment preview: the live calculator output plus a table showing the expected repayment schedule (Month 1: due date · \$100 · balance \$900, etc.).

Step 2 — Borrower submits

- POST /api/contracts — creates a draft contract with status 'pending_lender'.
- System generates: a lender invite link and one invite link per witness.
- Borrower's selfie (if taken) is uploaded and stored immediately.
- App displays all invite links with copy and Telegram share buttons.

Step 3 — Lender confirms

- Lender opens their invite link inside Telegram.
- App shows the full contract terms (read-only).
- Lender reviews, can edit their own pre-filled details (name, phone, etc.), optionally takes selfie.
- Lender taps 'Confirm and Sign' → POST /api/contracts/:uuid/lender-confirm.
- Contract status → 'pending_witnesses' (if witnesses exist) or 'active' (if no witnesses).

Step 4 — Witnesses confirm (optional, parallel)

- Each witness opens their unique link independently.
- Witness reviews contract, enters/confirms their details, optionally takes selfie.
- Taps 'I confirm as witness' → POST /api/contracts/:uuid/witness-confirm/:witness_index.
- Witness confirmation is NOT blocking — the contract activates when the lender confirms. Witnesses may confirm at any time after.
- PDF is regenerated automatically each time a new witness confirms, to include their updated info and selfie.

Step 5 — Contract activates + PDF generated

- On lender confirmation: contract status → 'active'.
- PDF is generated by the Node.js backend using Puppeteer.
- Bot sends Telegram notifications to borrower, lender, and any witnesses who have already confirmed.

1.7 Functional Requirements

i Tags: [M] = Must Have (MVP), [S] = Should Have (MVP), [D] = Defer to later phase.

FR-01 Authentication

- [M] Validate every request using Telegram initData HMAC-SHA256. Reject if hash mismatch or timestamp > 1 hour old.
- [M] On first visit: upsert user record from initData (telegram_id, first_name, last_name, username). Issue JWT.
- [M] Invite link recipients authenticate themselves when they open the link. The system knows which contract and which role they are filling.

FR-02 Contract Creation

- [M] Borrower fills contract form as described in Section 1.6.
- [M] Real-time months calculator: client-side JS, no API call, updates on every input event.
- [M] No interest field. No interest calculation. Total amount and monthly repayment amount are the only financial inputs.
- [M] Currency selector: UZS, USD, EUR.
- [M] Start date: date picker, defaults to today.
- [M] Optional free-text description / purpose of the loan (e.g. 'Car purchase', 'Medical expenses').
- [M] Each participant (borrower, lender, each witness) has an independent set of identity fields as per Section 1.4.
- [M] Selfie capture per participant: native camera input, optional, server-side resize to 400×400 px (sharp), stored at `contracts/{uuid}/photos/{role}.jpg`.
- [S] Borrower can save a draft and return later (status = 'draft').

FR-03 Contract Confirmation Flow

- [M] System generates unique invite tokens per role. Links: `https://t.me/{Bot}/app?startapp={uuid}_{role}_{token}`
- [M] Lender confirm endpoint: validate token, save lender's details and optional selfie, set status = 'active', trigger PDF generation.
- [M] Witness confirm endpoint: validate token and witness index (1, 2, or 3), save witness details and optional selfie, trigger PDF regeneration.
- [M] A participant can only confirm their own role. The system checks the JWT matches the invite token role.
- [S] Lender and witnesses can re-take selfie before confirming.

FR-04 PDF Generation

- [M] Generated using Puppeteer (headless Chrome). Triggered on lender confirmation. Regenerated on each witness confirmation.
- [M] PDF is A4 format, print-ready.
- [M] PDF structure:
 - Header: App name + logo, Contract UUID (full), creation date.
 - Title: 'Debt Agreement / Qarz Shartnomasi'
 - Debt Terms section: total amount, currency, monthly repayment, number of months, start date, description.
 - Repayment schedule table: Month #, due date, amount, cumulative paid, balance.
 - Participants section: one block per participant (Borrower, Lender, Witness 1/2/3). Each block contains: role label, first name, patronymic (if provided), last name, Telegram username, phone (if provided), address (if provided) — and the participant's selfie photo displayed directly to the right of this information. If no selfie: a grey placeholder silhouette.
 - Payment History section: table of all logged repayments (date, amount, notes, running balance). Empty on initial generation.
 - Disclaimer: 'This document is a formal record of mutual intent. Consult a qualified lawyer for legal enforcement.'
 - QR code at the BOTTOM of the last page, centered, with caption: 'Scan to view live repayment status and download updated PDF'.
- [M] PDF filename: `qarz_{uuid_short}_{date}.pdf`

- [S] PDF is generated in the language the borrower selected at contract creation. Language is stored on the contract record and used for all subsequent PDF regenerations (including public downloads).

FR-05 Repayment Logging

- [M] Lender logs a repayment when money is received: date (default today), amount, optional note.
- [M] System records the payment, recalculates remaining balance, and stores `running_balance` on the payment row.
- [M] Both the public status page and the downloadable PDF update to reflect the new payment.
- [M] If total paid $\geq$ total amount: contract status $\rightarrow$ 'settled'.
- [S] Borrower receives a Telegram bot notification when a repayment is logged by the lender.

FR-06 Public Status Page (QR Target)

- [M] URL: `https://yourdomain.com/public/status/:uuid` — no authentication, no login.
- [M] Page displays ALL of the following:
 - Contract UUID (partial for display, full in metadata)
 - All participant blocks: same layout as PDF (name, patronymic, username, phone, address, selfie if available)
 - Debt terms: total, monthly, number of months, currency, start date, description
 - Repayment progress: amount paid, amount remaining, months completed vs total, progress bar
 - Full repayment history table: each logged payment with date, amount, notes
 - Contract status badge: Active / Settled / Overdue
- [M] Download PDF button: calls `GET /public/status/:uuid/pdf` — regenerates PDF with all current data (including full payment history) and triggers browser download.
- [M] Page is mobile-responsive. Works in any smartphone browser after scanning the QR.
- [M] Page does NOT require the visitor to have Telegram or to be logged in.
- [S] Rate limit: max 120 requests/hour per IP (express-rate-limit, no Redis needed).

FR-07 Dashboard (Authenticated Users)

- [M] User sees all contracts where they are the borrower, lender, or a witness.
- [M] Contract cards show: counterparty name, role badge, status, remaining amount.
- [M] Tapping a card opens the contract detail (same data as public page, plus the ability to log repayments if lender).
- [S] Borrower can manually mark a contract as settled (prompts confirmation).

1.8 Data Model

Table	Key Fields	Notes
users	id, telegram_id, first_name, last_name, username, created_at	Auto-created on first Telegram login. No passwords.
contracts	uuid, borrower_user_id, total_amount, currency, monthly_amount, n_months,	language: 'uz' 'ru' 'en' — set by borrower at creation. Used for all

Table	Key Fields	Notes
	start_date, description, language, status, pdf_path, created_at, activated_at	PDF generation for this contract. status: draft pending_lender active settled overdue. No interest fields.
participants	id, contract_uuid, role (borrower lender witness1 witness2 witness3), invite_token, first_name, last_name, patronymic, telegram_username, phone, address, photo_path, confirmed_at, confirmed_by_user_id	One row per participant per contract. Selfie stored at photo_path (nullable). invite_token is the one-time confirmation token.
payments	id, contract_uuid, logged_by_user_id, amount, payment_date, note, logged_at, running_balance	Logged by LENDER when money is received. Immutable after insert. running_balance computed and stored for fast retrieval.
notifications	id, user_id, contract_uuid, type, sent_at	Queue for Telegram bot messages. Processed by a background job.

Key schema decisions:

- No interest_rate column anywhere. Intentionally omitted.
- participants table unifies borrower, lender, and witnesses into one structure, making the PDF layout logic uniform: iterate over participants in order, render each block identically.
- pdf_path on contracts always points to the LATEST generated PDF (overwritten on each regeneration).
- running_balance is stored on each payment row to avoid recalculating the full history on every page load.

1.9 Technology Stack — Final Specification

Layer	Technology	Rationale
Mini App frontend	React 18 + TypeScript + Vite	Standard Telegram Mini App stack. Fast HMR. Strong typing for financial fields.
Telegram SDK	@tma.js/sdk (official)	Official Telegram Mini Apps JS SDK. Actively maintained. Replaces older community wrappers.
Backend API	Node.js 20 + Express + TypeScript	Single language across entire stack. TypeScript prevents type errors in financial calculations.
Database	PostgreSQL 16 + Prisma ORM	ACID compliance required. Prisma provides type-safe queries and migration management.
Telegram Bot	telegraf v4	Sends notifications when contracts activate and repayments are logged.
PDF generation	Puppeteer (headless Chrome)	Renders an HTML template to PDF. Supports full CSS layout, images, and QR codes inline. No language conflicts.
QR code	qrcode (npm)	Generates PNG or data URL. Injected directly into the Puppeteer HTML template as a base64 .

Layer	Technology	Rationale
Photo capture	HTML <code><input type='file' accept='image/*' capture='user'></code>	Native OS camera dialog. Works on all platforms. No WebView camera API, no library dependency.
Image resizing	sharp (npm)	Resizes selfies to 400×400 px server-side. Lightweight, native bindings, fast.
UI components	Tailwind CSS + shadcn/ui	Telegram's dark/light theme CSS variables map naturally to Tailwind tokens.
File storage	Server local filesystem	Simple for MVP. Paths stored in DB. Migrate to MinIO / S3-compatible later if needed.
Auth	Telegram initData + JWT	HMAC validation then JWT for API sessions. 24h expiry.
Security headers	helmet (npm)	Content-Security-Policy, X-Frame-Options, etc.
Rate limiting	express-rate-limit (npm)	IP-based, in-memory. No Redis required for MVP.
Localisation	i18next + react-i18next	Three languages: Uzbek (uz), Russian (ru), English (en). Auto-detected from Telegram language_code. Manual override stored in user preferences.
Deployment	Ubuntu 24 VPS + Nginx + PM2	Simple, controllable, cost-effective. 2 vCPU / 4 GB RAM minimum (Puppeteer needs headroom).

1.10 API Endpoints — MVP

Method	Endpoint	Auth	Description
POST	/api/auth/telegram	Public	Validates initData, returns JWT.
GET	/api/me	JWT	Current user profile.
POST	/api/contracts	JWT	Create draft contract (borrower only).
GET	/api/contracts	JWT	List all contracts for current user.
GET	/api/contracts/:uuid	JWT	Full contract details. Only participants.
POST	/api/contracts/:uuid/photo	JWT	Upload selfie for the requesting participant.
POST	/api/contracts/:uuid/lender-confirm	Invite token	Lender confirms. Triggers status → active + PDF generation.
POST	/api/contracts/:uuid/witness-confirm/:n	Invite token	Witness n (1–3) confirms. Triggers PDF regeneration.
POST	/api/contracts/:uuid/payments	JWT	Lender logs a repayment received.
GET	/api/contracts/:uuid/payments	JWT	Payment history (authenticated).
GET	/public/status/:uuid	Public	Full contract data + all participants + payment history. No login.
GET	/public/status/:uuid/pdf	Public	Regenerate + download current PDF with full payment history.

1.11 PDF Layout Specification

The PDF is generated from an HTML template rendered by Puppeteer. The layout must be reproducible both at contract creation and on every public download (which includes updated payment history).

Section	Content
Header	App logo (left) · Contract UUID (right) · 'Debt Agreement / Qarz Shartnomasi' centered · creation date.
Debt Terms	Total amount · Currency · Monthly repayment · Number of months · Start date · Purpose/description (if provided).
Repayment Schedule	Table: Month # Due Date Payment Amount Balance After. Last row shows total.
Participants	One block per participant, in this order: Borrower → Lender → Witness 1 → Witness 2 → Witness 3. Each block is a two-column layout: LEFT: Role label (bold) · Full name · Patronymic · Telegram @username · Phone · Address · Confirmed: [date or 'Pending'] RIGHT: Selfie photo (400×400, cropped circle) or grey placeholder silhouette if no selfie.
Payment History	Table: # Date Amount Paid Note Running Balance. Shows 'No payments logged yet' if empty.
Disclaimer	'This document is a formal record of mutual intent between the parties named above. It does not constitute a notarized legal instrument. Consult a qualified lawyer for legal enforcement or dispute resolution.'
QR Code (bottom)	Centered on the last page. Size: 150×150 px. Caption below: 'Scan to view live status and download updated PDF'. URL encoded: <code>https://yourdomain.com/public/status/{uuid}</code>

i Puppeteer note: inject selfie photos as base64 data URLs directly in the HTML template. Do not use external file:// paths — Puppeteer sandbox may block them. Read photos with `fs.readFileSync` and convert to base64 before rendering.

1.12 Non-Functional Requirements

Requirement	Specification
PDF generation time	< 8 seconds per PDF. (Puppeteer cold start ~2s + template render ~1s.)
API response time	< 500 ms for all data reads under normal load.
App load time	< 3 seconds inside Telegram on 4G.
File upload limit	5 MB per photo. MIME type validated server-side with the file-type npm package.
Photo formats	JPEG, PNG, WEBP only. Resize to 400×400 using sharp before storage.
Auth tokens	JWT: 24h expiry. Invite tokens: single-use, expire after 30 days.
Data backup	pg_dump daily cron. 7-day retention. Test restore before launch.
Photo privacy	Selfie photos served on the public page. Participants are informed of this before taking a selfie. Warning: 'This photo will be visible to anyone who scans the QR code.'

Requirement	Specification
Puppeteer sandbox	Run with --no-sandbox on VPS. Alternatively: dedicated low-privilege user for the Puppeteer subprocess. Validate all template inputs to prevent HTML injection.
Rate limiting	Public endpoints: 120 req/hr per IP. Auth endpoints: 20 req/hr per IP. express-rate-limit, in-memory.

PART 2 — Step-by-Step Build Roadmap — NomachiBot v1.0

The roadmap is structured so each phase produces something testable before the next phase begins. This is version 1.0, built from zero. Solo developer estimate: 6–8 weeks. With an AI pair-programming agent: 3–4 weeks. Do not skip phases or merge them — each produces a testable checkpoint.

Phase 1 — Foundation (Days 1–5)

i Goal: Server reachable · database running · bot responds to /start · HTTPS live.

#	Task	Details
1.1	Provision VPS	Ubuntu 24 LTS. Min: 2 vCPU, 4 GB RAM (Puppeteer needs headroom). Install: Node 20 (via nvm), PostgreSQL 16, Nginx, PM2, Git, Certbot.
1.2	Create Telegram Bot	Message @BotFather: /newbot → get BOT_TOKEN. Then /newapp → configure Mini App URL. Store BOT_TOKEN in .env.
1.3	Backend repo init	mkdir nomachibot-api && npm init -y && npm install express typescript ts-node prisma @prisma/client jsonwebtoken telegraf multer puppeteer qrcode sharp helmet express-rate-limit dotenv file-type && npx tsc --init
1.4	Configure Postgres	Create database 'nomachibot'. Create role 'nomachibot_user' with password. Set DATABASE_URL in .env.
1.5	Prisma schema	Define all 5 tables from Section 1.8. Note: contracts table has NO interest_rate field. Run: npx prisma migrate dev --name init. Verify tables.
1.6	Hello World API	Express on port 3001. GET /health → { status: 'ok', timestamp }. Deploy with PM2. Nginx reverse proxy.
1.7	HTTPS certificate	certbot --nginx -d yourdomain.com. Telegram requires HTTPS for Mini Apps. Test: curl https://yourdomain.com/health.
1.8	Bot /start command	Telegraf: handle /start → send message with inline button opening Mini App URL. Run as PM2 process.

✓ Phase 1 complete when: HTTPS health check returns 200 and the bot opens the Mini App on /start.

Phase 2 — Authentication (Days 6–9)

i Goal: Any Telegram user who opens the Mini App gets a valid JWT. Invite token system is designed and tested.

#	Task	Details
2.1	Frontend repo init	npm create vite@latest nomachibot-app -- --template react-ts && npm install @tma.js/sdk axios react-router-dom tailwindcss i18next react-i18next react-hot-toast
2.2	Telegram SDK init	In App.tsx: import { initMiniApp, retrieveLaunchParams } from '@tma.js/sdk'. Call initMiniApp(). On mount: retrieve startParam — if it contains a UUID + role + token, redirect to the appropriate confirmation screen.

#	Task	Details
2.3	POST /api/auth/telegram	Backend: parse initData string, validate HMAC-SHA256 against BOT_TOKEN. Reject if hash wrong or auth_date > 3600s ago. Upsert user record. Return signed JWT (24h).
2.4	JWT middleware	Express middleware: extract Bearer token, verify with jsonwebtoken, attach decoded user to req.user. Apply to all /api/* except /public/*.
2.5	Invite token design	When a contract is created: generate a cryptographically random 32-byte invite token per role using crypto.randomBytes(32).toString('hex'). Store in participants table. Invite link format: https://t.me/{Bot}/app?startapp={uuid}_{role}_{token}
2.6	Test auth	Open Mini App in Telegram. Verify GET /api/me returns your Telegram profile.

✓ Phase 2 complete when: /api/me returns your Telegram user data inside the Mini App.

Phase 3 — Contract Creation UI (Days 10–16)

i Goal: Borrower can fill the full form, use the real-time calculator, optionally take a selfie, and submit a draft contract.

#	Task	Details
3.1	Telegram theme + design system	Map CSS variables: var(--tg-theme-bg-color), --tg-theme-text-color, --tg-theme-button-color → Tailwind tokens. Create base components: Button, Input, TextArea, Select, Card, Badge, PhotoCapture.
3.2	App routing	React Router: / → dashboard, /create → new contract form, /contract/:uuid → detail, /confirm/:uuid/:role/:token → confirmation screen (lender/witness). Back button uses WebApp.BackButton.
3.3	My Details section (Borrower)	Pre-populate first/last name and Telegram username from JWT. Fields: First name* (pre-filled, editable), Last name* (pre-filled, editable), Patronymic (optional), Phone (optional, +998 prefix default), Address (optional).
3.4	Lender Details section	Fields: First name*, Last name*, Telegram @username* (borrower types this), Patronymic, Phone, Address. Mark mandatory fields with *.
3.5	Witnesses section	Collapsible. 'Add Witness' button (up to 3). Each witness row: First name*, Last name*, @username* inputs. 'Remove' icon per witness. Witnesses are entirely optional — borrower can skip this section.
3.6	Debt Terms section	Currency selector (UZS / USD / EUR). Total Amount: number input (2 decimal places). Monthly Repayment: number input. No interest rate field — do not add one.
3.7	Real-time calculator	TypeScript function: calcMonths(total: number, monthly: number): number => Math.ceil(total / monthly). Call on every onChange of both inputs. Display result in a highlighted box: '10 months to repay' + 'Last payment: \$100.00'. Show '—' when monthly is 0 or empty.
3.8	Repayment preview table	Below the calculator: generate an array of { month, dueDate, amount, balance } client-side. Render as a scrollable table. Update with each keystroke.
3.9	Selfie capture (Borrower)	Optional section with camera icon. <input type='file' accept='image/*' capture='user'>. Show circular preview after capture. Show 'Retake'

#	Task	Details
		button. Show warning: 'This photo will be visible to anyone who scans the QR code.'
3.10	Submit to backend	POST /api/contracts (JSON: all identity fields, debt terms, witness names). Then, if selfie taken: POST /api/contracts/:uuid/photo (multipart). Show WebApp.MainButton spinner.
3.11	Backend: save contract	Validate: amounts > 0, monthly <= total, mandatory names present, Telegram usernames format valid (@handle). Create contract row (status='pending_lender'). Create participants rows (borrower + lender + witnesses). Generate invite tokens. Store borrower selfie if provided.
3.12	Show invite links	After creation: show each invite link with label (Lender, Witness 1, etc.), copy button, and Telegram share button (uses WebApp.openTelegramLink with a share URL). Trigger haptic success feedback.

✓ Phase 3 complete when: borrower submits form, contract appears in DB with status 'pending_lender', selfie stored, invite links displayed.

Phase 4 — Lender & Witness Confirmation (Days 17–22)

i Goal: Lender and witnesses can open their invite links, confirm, optionally take selfies, and activate the contract.

#	Task	Details
4.1	Detect invite link on open	On Mini App launch: parse startParam from @tma.js/sdk retrieveLaunchParams(). If pattern matches {uuid}_{role}_{token}: redirect to /confirm/:uuid/:role/:token.
4.2	Confirmation screen	Fetch contract summary. Display: role label, all debt terms, repayment schedule, borrower's name. Pre-fill the confirming participant's identity fields from their Telegram account. They can edit first/last name, add patronymic/phone/address.
4.3	Optional selfie on confirm	Same selfie capture component as Step 3.9. Show same QR visibility warning. Optional.
4.4	Lender confirm endpoint	POST /api/contracts/:uuid/lender-confirm with body: { token, first_name, last_name, patronymic?, phone?, address? }. Backend: validate token matches participants record for role='lender'. Save details. Save selfie if provided. Update status → 'active'. Trigger PDF generation.
4.5	Witness confirm endpoint	POST /api/contracts/:uuid/witness-confirm/:n. Same pattern. Does NOT change contract status (witnesses are non-blocking). Triggers PDF regeneration.
4.6	Success screen	After confirmation: show 'Confirmed!' screen with contract UUID, link to public status page, and 'Download PDF' button.

✓ Phase 4 complete when: lender opens invite link in Telegram, confirms, contract status becomes 'active' in DB.

Phase 5 — PDF Generation (Days 23–27)

i Goal: A correctly structured PDF is generated on lender confirmation and regenerated on each witness confirmation. The PDF matches the layout spec in Section 1.11.

#	Task	Details
5.1	HTML contract template	Create a Handlebars (.hbs) or EJS template file. Sections: header, debt terms, repayment schedule table, participants, payment history, disclaimer, QR code. All styled with inline CSS (Puppeteer renders inline CSS reliably).
5.2	Participant block layout	Each participant block: a two-column <div> with flexbox. Left: text info. Right: circular (photo as base64 data URL, or grey placeholder SVG if no photo). Width ratio: 70% / 30%. Iterate over participants array in order: borrower, lender, witness1, witness2, witness3.
5.3	QR code generation	<code>const qrDataUrl = await QRCode.toDataURL('https://yourdomain.com/public/status/' + uuid, { width: 150, margin: 2 })</code> . Inject as base64 src at the bottom of the template.
5.4	Photos as base64	Read each participant's photo with <code>fs.readFileSync(photoPath)</code> . Convert: <code>'data:image/jpeg;base64,' + buf.toString('base64')</code> . Inject into template. Never use <code>file://</code> URLs in Puppeteer — sandbox blocks them.
5.5	Puppeteer render	<code>const browser = await puppeteer.launch({ headless: 'new', args: ['--no-sandbox', '--disable-setuid-sandbox'] })</code> . Set page content with <code>await page.setContent(html, { waitUntil: 'networkidle0' })</code> . <code>await page.pdf({ path: outputPath, format: 'A4', printBackground: true })</code> . Close browser.
5.6	PDF service module	Wrap PDF logic in a PdfService class with: <code>generateContractPdf(uuid)</code> and a simple queue (max 2 concurrent jobs, using a semaphore). Prevents memory spikes on burst requests.
5.7	Store + serve PDF	Save to <code>/var/data/nomachibot/contracts/{uuid}/agreement.pdf</code> . Update <code>pdf_path</code> in contracts table. GET <code>/api/contracts/:uuid/pdf</code> (authenticated) and GET <code>/public/status/:uuid/pdf</code> (public) both serve this file with <code>res.sendFile()</code> . Public endpoint regenerates the PDF first (to include latest payments).
5.8	Test PDF output	Generate a test contract with: 2 participants with selfies, 1 witness without selfie, 2 logged payments. Verify PDF layout matches Section 1.11. Print physically and scan the QR code.

✓ Phase 5 complete when: PDF has all sections, QR is scannable, photos are next to participant info, placeholder appears for participants without selfies.

Phase 6 — Public Status Page (Days 28–32)

i Goal: Anyone who scans the QR code can view full contract data, all payments, and download an updated PDF — without logging in.

#	Task	Details
6.1	GET /public/status/:uuid	No auth. Returns full JSON: contract terms, all participants (name, phone, address, selfie URL if available), repayment schedule, full payment history, contract status. Rate limit: 120/hr per IP.
6.2	Selfie access on public page	Photos are shown on the public page because participants consented at capture (warning was shown). Serve via GET

#	Task	Details
		/public/photos/:uuid/:role — validates that the contract exists; no auth required.
6.3	Public page React route	Route: /public/status/:uuid. No JWT required. Fetches from /public/status/:uuid API. Shows: status badge, debt terms, progress bar, all participant blocks (same two-column photo + info layout as PDF), payment history table, Download PDF button.
6.4	GET /public/status/:uuid/pdf	No auth. Calls PdfService.generateContractPdf(uuid) (always regenerates to include latest payments). Returns file with Content-Disposition: attachment; filename=qarz_{uuid_short}_{date}.pdf
6.5	Mobile responsive design	The public page must render well on a phone browser. Use Tailwind responsive classes. Participant photos: circular, max 120px on mobile. Payment history table: horizontally scrollable on small screens.
6.6	End-to-end QR test	Print or display a generated PDF. Scan QR with a phone. Verify status page loads with correct data. Tap Download PDF. Open downloaded file and verify it matches the page content.

✓ Phase 6 complete when: scanning the QR opens the status page, Download PDF works, no login required.

Phase 7 — Payment Logging (Days 33–36)

i Goal: Lender can log repayments as they are received. Balance updates. Public page and PDF reflect each payment. Borrower is notified.

#	Task	Details
7.1	Contract detail screen	Tabs: Overview Schedule Payments. Overview: status badge, amounts, next due date. Schedule: all installments with status colours (paid=green, overdue=red, pending=grey). Payments: full history.
7.2	Log payment modal	Only shown to LENDER. Fields: Date (default today), Amount (default next installment amount), Note (optional). Submit → POST /api/contracts/:uuid/payments. Backend validates: requesting JWT belongs to the lender of this contract.
7.3	Backend: process payment	Validate: requesting user is the lender of this contract (check participants table), amount > 0, contract is active, date is not in the future. Insert payment row. Compute running_balance = previous_balance - amount. If running_balance <= 0: update contract status → 'settled', notify all participants.
7.4	Overdue detection	PM2 cron (daily 09:00 server time): find contracts where status='active' AND sum of payments for current period < expected. Mark overdue. Send bot notification to borrower and lender.
7.5	Bot notifications	On payment logged: bot sends message to BORROWER — 'Lender recorded a payment of [amount] on [date]. Remaining balance: [balance]'. On contract settled: notify all participants.

✓ Phase 7 complete when: lender logs a payment, balance updates on public page, borrower receives Telegram notification, PDF download reflects the payment.

Phase 8 — Dashboard, Polish & Localisation (Days 37–44)

#	Task	Details
8.1	Dashboard	Three tabs: Borrowing Lending Witnessing. Contract cards: counterparty name, role badge, status, remaining balance. Empty state with illustration. Tap → contract detail.
8.2	Theme integration	Read <code>WebApp.colorScheme</code> and <code>WebApp.themeParams</code> . Call <code>WebApp.setHeaderColor()</code> and <code>WebApp.setBackgroundColor()</code> . Full dark/light mode support.
8.3	Haptic feedback	<code>WebApp.HapticFeedback</code> : impact on button presses, notification on confirmation, error on validation failures, selection on tab switches.
8.4	Three-language localisation	i18next: create translation files for <code>uz.json</code> , <code>ru.json</code> , <code>en.json</code> . Translate all UI strings, error messages, and PDF template labels into all three. Language detection order: (1) user's saved preference, (2) Telegram <code>WebApp.language_code</code> , (3) fallback to English. Add a visible language switcher (UZ / RU / EN) in the app header — does not require re-login. PDF language is set at contract creation and stored in the contracts table (language column: <code>'uz' 'ru' 'en'</code>). The public page and downloaded PDFs always use the language stored on the contract.
8.5	Error handling	Axios interceptor catches all API errors → react-hot-toast notification in the active language. Never show raw error strings. Log errors server-side with context (uuid, user_id, route).
8.6	Manual QA	Full journey test on Android + iOS Telegram: borrower creates contract with 1 witness → share lender link → lender confirms on different device → witness confirms → download PDF → scan QR → lender logs payment → QR page updates → borrower receives notification → download updated PDF.

✓ Phase 8 complete when: full journey works on both platforms, language switcher works for all three languages, dark and light themes work correctly.

Phase 9 — Security Hardening & Launch (Days 45–50)

#	Task	Details
9.1	Environment audit	All secrets in <code>.env</code> (never committed). Use <code>dotenv-safe</code> to enforce required vars at startup. <code>.gitignore</code> includes <code>.env</code> .
9.2	File upload security	<code>multer</code> + <code>file-type</code> package: validate MIME type from file bytes (not extension). 5 MB limit. Store outside webroot. Serve via dedicated endpoint, never by guessable static URL.
9.3	Invite token expiry	Invite tokens expire after 30 days or first use (whichever comes first). Store <code>used_at</code> in participants table. Reject already-used tokens.
9.4	Helmet + CORS	<code>app.use(helmet())</code> . CORS: allow only your Mini App domain and Telegram domains. Never origin: <code>'*'</code> in production.
9.5	HTML injection prevention	Escape all user-supplied strings before injecting into Puppeteer HTML template. Use a templating engine (Handlebars auto-escapes). Never use <code>innerHTML</code> with user content.
9.6	Database backups	<code>pg_dump</code> daily cron to <code>/var/backups/qarzbot/</code> . 7-day retention. Run a manual restore test before going live.
9.7	Monitoring	UptimeRobot (free): monitor <code>/health</code> every 5 minutes. Alert if down. PM2 logs with daily rotation. Disk usage alert via cron if <code>> 80%</code> .

✓ Phase 9 complete when: security checklist passes, backups verified, monitoring active, app is live.

Post-MVP Roadmap — Phases 2 & 3

Start these only after the MVP is live and being actively used. Do not build them in advance.

Phase 2 — Biometric Hardening (After 100+ active users)

1. Deploy CompreFace on the same VPS via Docker Compose.
2. At confirmation: send each participant's selfie to CompreFace for face vector extraction. Store the vector in participants table.
3. At payment logging: prompt a quick selfie from the LENDER. Compare against stored vector (CompreFace /recognize). Reject if confidence < 90%.
4. Add MediaPipe Face Mesh (Google, actively maintained, browser-compatible) for simple liveness: require one blink before accepting selfie.
5. Add automated Telegram bot reminders: notify borrower 3 days before each installment due date.

Phase 3 — AI Document Parsing (After 500+ active users)

6. Add collateral attachment upload (PDFs, photos) to active contracts.
7. Deploy PaddleOCR as a Python Docker sidecar. Expose a single /ocr POST endpoint.
8. On attachment upload: send to OCR → extract text → store in contract_attachments table with full-text search index.
9. Show extracted key information in the contract detail screen.

Appendix — Quick Reference

A.1 npm Packages (Final MVP List)

Package	Version	Purpose
express	^4.19	Web framework
prisma + @prisma/client	^5.x	Database ORM + migrations
telegraf	^4.x	Telegram bot framework
@tma.js/sdk	latest	Official Telegram Mini App SDK (frontend)
jsonwebtoken	^9.x	JWT auth tokens
puppeteer	^22.x	PDF generation from HTML template
qrcode	^1.5	QR code PNG / data URL generation
sharp	^0.33	Selfie photo resizing to 400×400 px
multer	^1.4	Multipart file upload handling
file-type	^19.x (ESM)	Server-side MIME type validation for uploads
handlebars	^4.x	HTML template engine for PDF (auto-escapes user content)
helmet	^7.x	HTTP security headers
express-rate-limit	^7.x	IP-based rate limiting (in-memory, no Redis)
dotenv + dotenv-safe	^16.x	Environment variable management
i18next + react-i18next	^23.x	Three-language localisation: Uzbek (uz), Russian (ru), English (en)
react-hot-toast	^2.x	Toast notifications in Mini App
crypto (Node built-in)	built-in	Generate invite tokens: crypto.randomBytes(32).toString('hex')

A.2 Environment Variables

Variable	Description
DATABASE_URL	PostgreSQL connection string
BOT_TOKEN	Telegram Bot token from @BotFather
JWT_SECRET	Random 256-bit string. Generate: openssl rand -hex 32
MINI_APP_URL	Your Mini App public URL (for invite links)
PUBLIC_BASE_URL	Your API / public domain (for QR code URLs)
STORAGE_PATH	Absolute path for file storage, e.g. /var/data/nomachibot
PORT	API server port (default 3001)

A.3 Key Business Rules — Summary

Rule	Specification
Who initiates the contract	BORROWER only
Who activates the contract	LENDER confirmation activates it
Who logs repayments	LENDER only — records payments as they are received from the borrower
Interest rate	NONE. Not calculated, not stored, not displayed anywhere.
Months formula	$\text{Math.ceil}(\text{total_amount} / \text{monthly_amount})$. Client-side, real-time.
Mandatory identity fields	First name, Last name, Telegram ID for every participant.
Optional identity fields	Patronymic, phone, address, selfie — for every participant.
Maximum witnesses	3 witnesses. All optional.
Witness blocking	Witnesses do NOT block contract activation. Lender confirmation is sufficient.
PDF selfie placement	Each participant's selfie is displayed directly beside their info block. Placeholder if no selfie.
QR code placement in PDF	AT THE BOTTOM of the last page. Not in the header.
Public access	Anyone with QR sees all contract data and payments. No login required.
Contract language	Set by borrower at creation (uz / ru / en). Stored in <code>contracts.language</code> . All PDF regenerations — including public downloads — use this language.
App language	Set by user preference, detected from Telegram <code>language_code</code> , or chosen via the UZ / RU / EN switcher in the header. Saved to <code>localStorage</code> . Independent from contract language.
PDF download	Available from public page. Regenerated on every download to include latest payments.

A.4 Known Risks and Mitigations

Risk	Probability	Mitigation
Camera input broken on older iOS/Telegram	Medium	Native <code><input capture='user'></code> bypasses WebView camera API. Tested on iOS 16+.
Puppeteer memory spike on PDF burst	Medium	Semaphore: max 2 concurrent PDF jobs. Add 1 GB swap on VPS.
User uploads malicious file as selfie	High	Validate MIME type via file-type (reads magic bytes, not extension). Allow only jpeg/png/webp.
HTML injection via user-supplied fields into PDF	Medium	Handlebars auto-escapes all <code>{{variable}}</code> expressions. Never use triple-brace <code>{{{variable}}}</code> .
Invite token reuse attack	Low	Tokens are single-use. Mark <code>used_at</code> on first use. Reject any subsequent use.

Risk	Probability	Mitigation
Disk fills with PDFs and photos	Low (initially)	~500 KB per contract. 10 000 contracts $\approx$ 5 GB. Monitor disk via cron alert at 80% of capacity.
Legal challenge on contract validity	Medium	PDF disclaimer states this is a record of intent. Do not market as a notarized document.